

Date: _____ Day: (MTWTFSS) _____

Q Write various issues in the design of code generator?

The following issues arising during the code generation phase:

Input to code generator:

⇒ The input to code generation consists of the intermediate representation of the source program produced by front end, together with information in the symbol table to determine run-time addresses of the data objects denoted by the name in the intermediate representation.

⇒ Intermediate representation can be:

- o Linear representation such as postfix notation.
- o Three address representation such as quadruples.

Date: _____ Day: (MTWTFSS) _____

o Virtual machine representation such as stack machine code.

o Graphical representation such as syntax trees and dags.

⇒ Prior to code generation, the front end must be scanned, parsed and translated into intermediate representation along with necessary type checking.

Therefore, input to code generation is assumed to be error free.

2. Target Program:

⇒ The output of the code generator is the target program. The output may be:

(a) Absolute machine language:

- o It can be placed in a fixed memory location and can be executed immediately.

Date: _____

Day: MTWTF

(b) Relocatable machine languages:

- It allows subprograms to be compiled separately.

- It makes the process of code generation easier.

(c) Assembly language:

- We can generate symbolic instructions and use the macro facilities of assemblers in generating code.

⇒ However, out of these three forms of output target code it is always preferable to have relocatable machine code as target code.

(3) Memory management:

⇒ Names in the source program are mapped to addresses of data objects in run-time memory by the front-end and code generator.

Date: _____

Day: MTWTF

⇒ It makes use of symbol table, that is, a name in a three-address statement refers to a symbol table entry for the name. Labels in three-address statements have to be connected to addresses of instructions.

for example,

$j: goto\ i$ generates jump instruction as follows:

→ if $i < j$, a backward jump instruction with target address equal to location of code for quadruple i is generated.

→ if $i > j$, the jump is forward. We must store on a list for quadruple i the location of the first machine instruction generated for quadruple j . When i is processed, the machine location for all

Date: _____

Day: M T W T F S

instructions that forward jumps to i are filled.

4) Instruction selection:

⇒ The instructions of target machine should be complete and uniform.

⇒ Instruction speeds and machine idioms are important factors when efficiency of target program is considered.

⇒ The quality of the generated code is determined by its speed and size.

$a = b + c$
 $d = a + e$

can be translated into

MOV b, R_0

ADD c, R_0

MOV R_0, a

MOV a, R_0

ADD e, R_0

MOV R_0, d

— This can be eliminated

Date: _____

Day: M T W T F S

(5) Register allocation:

⇒ Instructions involving register operands are shorter and faster than those involving operands in memory.

⇒ The use of registers is subdivided into two subproblems:

Register allocation: (what value to hold in what reg?)

◦ The set of variables that will reside in registers at a point in the program is selected.

Register assignment:

◦ The specific register that a variable will reside in is picked.

⇒ Certain machine requires even-odd register pairs for some operands and results.

Date: _____

Day: MTWTF

for example, consider the division instruction of the form:
 D, x, y

(b) Evaluation Order:

- o The order in which the computations are performed can affect the efficiency of the target code.

Some computation orders require fewer registers to hold intermediate results than others.

Approaches to code generation

Issues:

⇒ Some of the design goals of code generators are:

- o Compact
- o Testable
- o Efficient
- o Easily maintainable

Date: _____

Day: MTWTF

Q. Generate the object code for:
 $a = b + c; \quad d = a + c$